



**Instituto Tecnológico Superior De
Rio verde**

Alumno: Luis Alejandro Domínguez Ramírez

Materia: Programación orientada a objetos

Profesor: ING. José de Jesús Collazo Reyes

Grupo: 2-A

Temas: Sistema de combate con
polimorfismo

Fecha: 05/05/2026

INTRODUCCION

En esta práctica se trabaja el concepto de polimorfismo dentro de la Programación Orientada a Objetos en C++. El polimorfismo es la capacidad que tienen distintos objetos de responder de manera diferente a una misma acción o método, dependiendo del tipo de objeto que sea. Esto permite que un mismo nombre de función, como “atacar”, pueda comportarse de forma distinta si lo ejecuta un guerrero, un mago o un arquero.

Este concepto es muy importante porque facilita la creación de programas más organizados, flexibles y reutilizables. En lugar de usar múltiples condicionales para identificar el tipo de objeto, el polimorfismo permite que el propio objeto decida qué acción realizar en tiempo de ejecución. Esto se logra mediante el uso de funciones virtuales y el uso de punteros o referencias a una clase base.

En el contexto de esta práctica, se simula un sistema de combate en un videojuego donde existen distintos tipos de personajes. Aunque todos pueden realizar la acción de atacar, cada uno lo hace de manera diferente. Gracias al polimorfismo, el programa puede manejar a todos los personajes de forma general sin necesidad de saber exactamente qué tipo de personaje es, permitiendo así ejecutar la misma instrucción y obtener comportamientos distintos.

En conclusión, el polimorfismo sirve para simplificar el código, hacerlo más escalable y permitir la extensión del sistema sin necesidad de modificar su estructura principal, lo cual es especialmente útil en el desarrollo de videojuegos y sistemas complejos.

CODIGO FUENTE

Personaje.h

```
C: Personaje.h > ...
1  #ifndef PERSONAJE_H
2  #define PERSONAJE_H
3
4  #include <iostream>
5  using namespace std;
6
7  // CLASE BASE
8  class Personaje {
9  protected:
10     string nombre;
11     int nivel;
12     int vida;
13
14 public:
15     Personaje(string n, int niv, int v);
16
17     void mostrarInfo();
18
19     virtual void atacar();
20 };
21
22
23 class Guerrero : public Personaje {
24 public:
25     Guerrero(string n, int niv, int v);
26     void atacar() override;
27 };
28
29 class Mago : public Personaje {
30 public:
31     Mago(string n, int niv, int v);
32     void atacar() override;
33 };
34
35 class Arquero : public Personaje {
36 public:
37     Arquero(string n, int niv, int v);
```

El archivo .h se utiliza para definir la estructura del programa. En este archivo se declaran las clases, sus atributos y los métodos que van a tener, pero sin incluir la implementación completa de estos.

En esta práctica, el archivo Personaje.h contiene la definición de la clase base Personaje y de las clases derivadas Guerrero, Mago y Arquero. Aquí se especifican los atributos como nombre, nivel y vida, así como los métodos como mostrarInfo() y atacar(), donde este último se declara como virtual para permitir el uso del polimorfismo.

La función principal del archivo .h es servir como una guía o contrato que indica qué puede hacer cada clase, mientras que la lógica de cómo lo hace se desarrolla en el archivo .cpp. Esto ayuda a mantener el código organizado, más fácil de entender y de mantener.

Personaje.cpp

```
Personaje.cpp > atacar()
1  #include "Personaje.h"
2
3
4
5  Personaje::Personaje(string n, int niv, int v) {
6      nombre = n;
7      nivel = niv;
8      vida = v;
9  }
10
11 void Personaje::mostrarInfo() {
12     cout << "Nombre: " << nombre << endl;
13     cout << "Nivel: " << nivel << endl;
14     cout << "Vida: " << vida << endl;
15 }
16
17 void Personaje::atacar() {
18     cout << nombre << " hace un ataque basico." << endl;
19 }
20
21
22
23 Guerrero::Guerrero(string n, int niv, int v)
24     : Personaje(n, niv, v) {}
25
26 void Guerrero::atacar() {
27     cout << nombre << " lanza un ATAQUE FUERTE con espada." << endl;
28 }
29
30
31
32 Mago::Mago(string n, int niv, int v)
33     : Personaje(n, niv, v) {}
34
35 void Mago::atacar() {
36     cout << nombre << " usa un HECHIZO MAGICO." << endl;
37 }
```

El archivo .cpp es donde se implementan los métodos que fueron declarados en el archivo .h. Es decir, aquí se define cómo funcionan realmente las funciones del programa.

En esta práctica, Personaje.cpp contiene la lógica de los métodos como mostrarInfo () y atacar (). Además, se implementa el comportamiento específico de cada clase derivada (Guerrero, Mago y Arquero), donde cada uno redefine el método atacar () de manera distinta. Esto permite que, al ejecutar el programa, cada personaje responda de forma diferente, aplicando así el polimorfismo.

Main.cpp

```
#include <iostream>
#include <vector>
#include "Personaje.h"

using namespace std;

int main() {
    vector<Personaje*> personajes;

    personajes.push_back(new Guerrero("Thor", 10, 100));
    personajes.push_back(new Mago("Merlin", 8, 80));
    personajes.push_back(new Arquero("Legolas", 9, 90));

    cout << "=== PERSONAJES ===" << endl;

    for (int i = 0; i < personajes.size(); i++) {
        personajes[i]->mostrarInfo();
        cout << "-----" << endl;
    }

    cout << "\n=== ATAQUES ===" << endl;

    for (int i = 0; i < personajes.size(); i++) {
        personajes[i]->atacar(); // 🔥 POLIMORFISMO
    }

    for (int i = 0; i < personajes.size(); i++) {
        delete personajes[i];
    }

    return 0;
}
```

El archivo main.cpp es donde se ejecuta el programa principal. Aquí se crean los objetos de los personajes y se controla el flujo del sistema.

En esta práctica, en el main se crean diferentes personajes como Guerrero, Mago y Arquero, y se almacenan en un vector de punteros a la clase base Personaje. Después, se recorre la colección para mostrar la información de cada personaje y ejecutar el método atacar ().

Gracias al polimorfismo, aunque todos los objetos se manejan como tipo Personaje, cada uno ejecuta su propio comportamiento al atacar, lo que demuestra que el sistema puede trabajar con distintos tipos de personajes de forma genérica.

CAPTURAS DE EJECUCION

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\LUIS\OneDrive\Documentos\Luisale29\luis12\001_Practicas\Semana 12> g++ main.cpp Personaje.cpp -o borrame
PS C:\Users\LUIS\OneDrive\Documentos\Luisale29\luis12\001_Practicas\Semana 12> ./borrame
=== PERSONAJES ===
Nombre: Thor
Nivel: 10
Vida: 100
-----
Nombre: Merlin
Nivel: 8
Vida: 80
-----
Nombre: Legolas
Nivel: 9
Vida: 90
-----

=== ATAQUES ===
Thor lanza un ATAQUE FUERTE con espada.
Merlin usa un HECHIZO MAGICO.
Legolas dispara una FLECHA.
PS C:\Users\LUIS\OneDrive\Documentos\Luisale29\luis12\001_Practicas\Semana 12> |
```

Este resultado es la ejecución de tu programa en C++ sobre un sistema de combate con polimorfismo.

Primero, en la terminal compilaste los archivos con `g++ main.cpp Personaje.cpp -o bórrame` y luego ejecutaste el programa con `./bórrame`.

Después, el programa imprime una lista de personajes:

- Thor (nivel 10, vida 100)
- Merlin (nivel 8, vida 80)
- Legolas (nivel 9, vida 90)

Esto representa que tienes distintos objetos de tipo personaje con atributos diferentes.

Luego aparece la sección de ataques:

- Thor usa un ataque fuerte con espada
- Merlin usa un hechizo mágico
- Legolas dispara una flecha

Aquí se ve lo importante del programa: el polimorfismo, porque aunque todos son “personajes”, cada uno ejecuta el método de ataque de forma diferente según su tipo.

EXPLICACION

En esta práctica, el polimorfismo se implementó mediante una clase base llamada Personaje y varias clases derivadas como Guerrero, Mago y Arquero. Todas comparten el mismo método atacar (), pero cada una lo redefine para tener un comportamiento diferente. Esto permite que el programa ejecute la misma instrucción y obtenga resultados distintos dependiendo del objeto.

Una función virtual es un método declarado en la clase base que puede ser sobrescrito en las clases derivadas. En este caso, atacar () se declara como virtual para que cada personaje pueda modificar su comportamiento al momento de ejecutarlo.

El uso de punteros a la clase base permite almacenar distintos tipos de objetos en una misma estructura, como un vector de Personaje*. La ventaja de esto es que el programa puede manejar todos los personajes de forma genérica sin necesidad de saber su tipo específico, lo que hace el código más flexible y fácil de extender.

CONCLUSIONES

En esta práctica aprendí cómo funciona el polimorfismo en la Programación Orientada a Objetos y cómo se puede aplicar en C++ para crear sistemas más flexibles y organizados. También comprendí la importancia de las funciones virtuales y el uso de punteros a la clase base para permitir que distintos objetos respondan de manera diferente a la misma acción.

Una de las principales dificultades fue entender cómo se relacionan la herencia y el polimorfismo, así como la correcta estructura del código usando clases base y derivadas. Sin embargo, con la práctica fue más fácil comprender su funcionamiento.

Este conocimiento es útil en la vida real porque permite desarrollar sistemas más escalables, especialmente en videojuegos y aplicaciones grandes, donde es necesario manejar muchos tipos de objetos de manera general sin complicar el código.